

PROOFLAYER

Product + Technical Brief

Evidence to onchain truth.

ProofLayer turns messy real-world evidence into an explainable verdict and a portable attestation on X Layer.

INTERNAL TEAM EDITION

Target build completion: 19 August 2026

Project path: /home/terry/Terrylinux_Workspce/prooflayer

This document explains the product for both technical and nontechnical teammates: what we are building, who it serves, how the system works, what is already implemented, what remains before the real launch, and how we will demonstrate it.

01 / THE PRODUCT

Executive summary

ProofLayer is trust infrastructure for X Layer applications that depend on facts outside the blockchain. A user submits evidence such as photos, PDFs, URLs, or structured data. ProofLayer keeps the source files private, fingerprints the evidence, runs deterministic policy checks plus an AI-assisted assessment, produces an explainable verdict, and anchors compact proof metadata to X Layer.

The first demonstration is a solar installation claim. The system receives a project statement, an installation photograph, and an inspection document. It evaluates whether the evidence supports the claim, returns VERIFIED, REJECTED, or NEEDS_REVIEW, records evidence and result hashes, and publishes an attestation that another application or person can inspect.

In plain language: ProofLayer is like a building inspector plus a tamper-proof receipt. The inspector evaluates the evidence. The receipt proves what was evaluated, which policy was used, who verified it, and whether the result has been revoked or expired.

What ProofLayer is: an evidence intake, verification, audit, and attestation layer.

What ProofLayer is not: a promise that AI knows the truth, a public file-sharing service, a replacement for human judgment in every case, or a token speculation product. Confidence is a signal; evidence, policy, limitations, verifier identity, and the onchain record remain inspectable.

02 / WHY THIS MATTERS

The problem we are solving

Blockchains record digital events well: a transaction was signed, a token moved, or a contract state changed. They are not naturally able to determine whether a physical installation happened, whether a delivery arrived, whether a document is genuine, or whether a milestone was completed in the real world.

X Layer applications therefore face an evidence gap. They may need to release a reward, approve a loan, issue a grant payment, mint an asset, validate a DePIN deployment, or accept a marketplace claim. Today those decisions are often based on screenshots, PDFs, private spreadsheets, manual review, or trust in one operator.

Existing approach	Weakness	ProofLayer response
Manual review	Slow, inconsistent, and difficult to audit at scale.	Structured policy checks, evidence hashes, and an audit trail.
Screenshots and PDFs	Can be edited, duplicated, or detached from the decision they supported.	Private source files plus tamper-evident digests and result hashes.
AI-only judgment	May hallucinate, overstate confidence, or hide limitations.	AI constrained by policy, validated output, limitations, and verifier provenance.
Onchain data only	Cannot represent many physical-world facts by itself.	Bring offchain evidence into an inspectable onchain attestation without publishing raw files.

Core thesis: make real-world claims legible to smart contracts without putting private evidence onchain.

03 / AUDIENCE AND VALUE

Who the product serves

Our primary customer is not every retail OKX user. The primary customer is an X Layer builder or business whose application needs trustworthy evidence. The OKX Wallet user is the end user who consumes and checks the proof before trusting a claim or signing a transaction.

Audience	Their point of view	What ProofLayer gives them
X Layer builders	My dApp depends on facts the chain cannot see.	An API and contract layer that converts evidence into portable attestations.
Businesses and claim submitters	I need to prove that an event or milestone happened.	Private evidence intake, an explainable verdict, and a reusable proof record.
Verifiers and auditors	Manual review is expensive and hard to reproduce.	A constrained pipeline, policy versioning, verifier identity, and audit history.
DAOs, lenders, marketplaces	I need confidence before releasing money, access, or rewards.	A machine-readable decision signal with evidence and limitations attached.
OKX Wallet users	I want to know whether a claim is credible before I interact.	A public proof page with verdict, hashes, verifier, policy, and X Layer transaction.
Public observers	I want to check a claim without receiving private files.	A transparent attestation record without exposing source documents.
Hackathon judges	I need real AI and real blockchain utility.	A complete path from messy evidence to a live X Layer attestation.

Initial wedge: physical-world milestone verification, beginning with solar installation completion. Later verticals may include DePIN hardware deployment, RWA documentation, grant milestones, delivery confirmation, climate claims, and marketplace authenticity.

04 / HOW TO EXPLAIN IT

The nontechnical explanation

A blockchain can remember that a transaction happened, but it cannot see the real world. ProofLayer checks real-world evidence with AI and clear rules, keeps the original files private, and records the result on X Layer so other people can verify that the decision was made and has not been secretly changed.

Short version: ProofLayer gives smart contracts eyes.

Analogy: ProofLayer is a building inspector plus a tamper-proof receipt. The inspector evaluates the evidence. The receipt records the claim, evidence fingerprint, policy, verifier, result, and onchain timestamp.

Important nuance: ProofLayer does not say, 'AI is always right.' It says, 'Here is what was submitted, how it was evaluated, which checks passed or failed, what the system could not determine, and what result was anchored.'

05 / ONE PROOF MOMENT

The end-to-end product flow

The product is a sequence of controlled transitions. Each transition creates a record that can be inspected later.

1. CREATE CLAIM

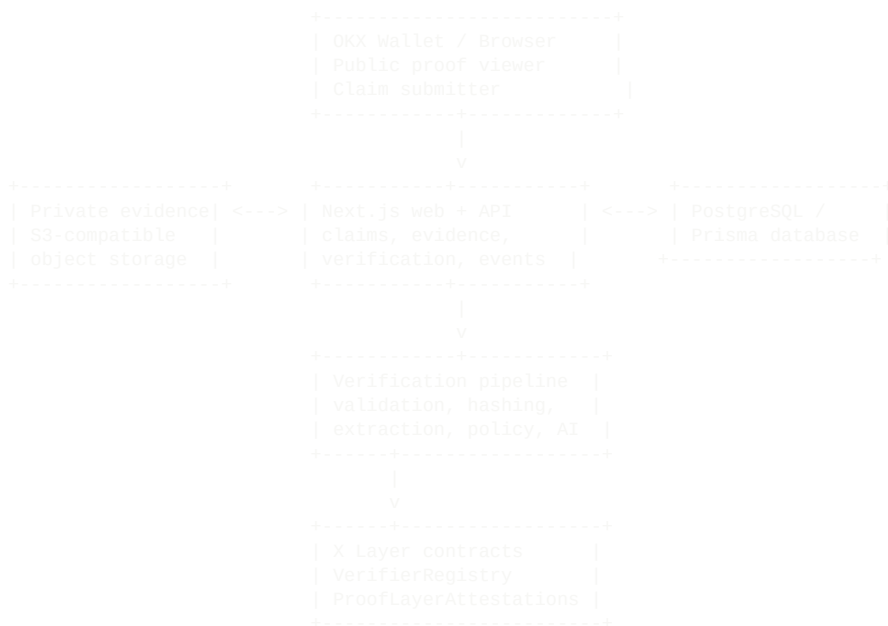
1. A submitter describes the claim and selects a policy.
2. UPLOAD EVIDENCE
Photos, PDFs, URLs, and JSON are accepted within validation limits.
3. PROTECT AND FINGERPRINT
Originals go to private storage. Each item receives a SHA-256 digest.
4. EXTRACT FACTS
The provider extracts text, metadata, visual observations, and document facts.
5. APPLY POLICY
Deterministic checks and AI assessment produce checks, confidence, limitations, and a verdict: VERIFIED, REJECTED, or NEEDS_REVIEW.
6. SAVE THE RESULT
The database records the run, model version, policy version, hashes, evidence, audit events, and verifier profile.
7. ANCHOR TO X LAYER
A registered verifier operator publishes compact attestation metadata onchain.
8. INSPECT OR ACT
A public proof page shows the result. A dApp may use it for decisions.
9. CHALLENGE OR REVOKE
A claim can be challenged offchain and an attestation revoked onchain.

Solar example: 'The solar installation at Project PL-1042 was completed.' Evidence includes an installation photograph and inspection PDF. The policy asks whether the evidence contains required installation indicators, whether the inspection supports completion, and whether the evidence set is internally consistent. The proof page shows the verdict and reasoning summary, not the private originals.

06 / TECHNICAL OVERVIEW

System architecture

ProofLayer is a modular TypeScript monorepo. The frontend and API are currently in one Next.js application for speed. The verifier, database, and contracts are isolated packages so the runtime can evolve without rewriting the product.



Boundary: private files stay offchain. The database stores operational metadata and history. The chain stores the minimum compact record needed to verify the attestation.

Current boundary: UI and API work in fixture/in-memory mode. The database schema, contracts, and provider seam exist, but real PostgreSQL, storage, Groq, and X Layer testnet deployment remain integration work.

Technical stack and repository map

Layer	Technology	Responsibility
Web experience	Next.js App Router, React, TypeScript, CSS	Landing page, workspace, claim form, result page, public proof page.
API	Next.js route handlers	Claim lifecycle, evidence intake, verification, events, challenge, revoke, health.
Verification	TypeScript package	Validation, SHA-256 hashing, extraction adapter, policy evaluation, result construction.
AI provider	Provider-agnostic adapter; Groq target	Vision/text extraction and structured assessment.
Database	PostgreSQL + Prisma	Claims, evidence, runs, attestations, verifiers, audit events, challenges.
Storage	S3-compatible private object store; R2 target	Private originals and controlled temporary access.
Blockchain	Solidity 0.8.24, OpenZeppelin, Hardhat	Verifier registry and attestation lifecycle on X Layer.
Chain operations	EVM RPC and relayer wallet	Deployment, registration, and attestation transactions.

prooflayer/
 apps/web/
 app/
 lib/store.ts
 packages/verifier/
 packages/contracts/
 packages/db/
 docs/

Next.js product and API
pages and route handlers
current runtime store seam
validation, hashing, policy, provider seam
Solidity contracts, deploy script, tests
Prisma schema, migration, repository, storage
implementation plan and worktree documentation

Important routes include: `/app`, `/app/claims/new`, `/app/claims/[claimId]`, `/proof/[claimId]`, and API routes for claims, evidence, verification, events, policies, verifiers, challenges, revocation, and health.

AI and verification pipeline

The AI is not the whole verifier. The pipeline separates evidence handling, deterministic validation, AI-assisted extraction or assessment, policy logic, result validation, storage, and onchain anchoring. This lets us change models without changing the policy or contract interface.

Stage	What happens	Why it exists
Input validation	Require item, unique IDs, supported kinds, URL presence, and a 10 MB per-item limit.	Reject malformed or unsafe input before model or storage work.
Evidence hashing	Hash exact bytes used for the run with SHA-256.	Bind the result to submitted evidence without publishing the file.
Extraction	Normalize text, metadata, visual observations, and document facts.	Give policy a consistent representation.
Assessment	Provider returns verdict, confidence, checks, limitations, and model version.	Use AI only through a strict output contract.
Policy result	Apply selected policy and construct canonical result hash.	Make decisions versioned and reproducible.

Stage	What happens	Why it exists
Persistence	Store run, evidence digests, result, and audit events.	Keep history for review and reprocessing.
Attestation	Publish compact metadata through registered verifier operator.	Make the decision portable and inspectable.

Recommended Groq setup: use qwen/qwen3.6-27b for multimodal extraction because it supports image and text inputs, OCR, document understanding, reasoning, and JSON output. Use openai/gpt-oss-120b for final policy reasoning because Groq lists it as a production model with a large context window and reasoning capability. Qwen is currently marked preview, so every run must record the model and a fallback must exist.

Output: verdict, confidence from 0 to 100, model version, policy version, checks, limitations, evidence digests, result hash, and verifier identity. Invalid output is rejected rather than silently accepted.

Safety: evidence is untrusted input. A document or image cannot override the system policy, request secrets, or change the verdict schema. The model returns facts and reasoning inside the allowed JSON shape; code decides whether the result can continue.

09 / WHAT STAYS PRIVATE

Privacy and evidence model

Photos and documents may contain addresses, names, signatures, serial numbers, or business information. ProofLayer should never put raw evidence or full private URLs onchain.

Data	Location	Visibility
Original photo or PDF	Private object storage	Claim owner and authorized service only.
Storage key	Database	Backend-controlled; not a public URL.
Evidence SHA-256	Database and attestation	Publicly verifiable fingerprint, not the original file.
Claim text	Database; selected public fields	Public only when the claim is shared.
AI assessment	Database and proof page summary	Explainable summary; secrets and raw files excluded.
Result hash	Database and X Layer	Public proof of the canonical result.
Verifier identity	Database and X Layer	Public provenance.
Transaction hash	Database and explorer	Public chain confirmation.

Uploads should use server-issued presigned URLs or an equivalent short-lived mechanism. The browser uploads directly to the private bucket, while the backend controls which object key is valid for which claim. Download links expire and are not permanent public proof.

10 / X LAYER ATTESTATION LAYER

Smart contract design

The contracts are intentionally narrow. They do not store raw evidence, run AI, or make subjective judgments. They register who is allowed to attest and store the compact outcome of an offchain verification run.

Contract	Purpose	Important behavior
VerifierRegistry	Registers verifier identities and operator addresses.	Owner registers verifier ID, operator, manifest hash, and pipeline version; verifier can be activated or deactivated.

Contract	Purpose	Important behavior
ProofLayerAttestations	Stores one attestation per claim ID.	Requires an active verifier operator, validates confidence and verdict, supports expiry, read access, revocation, pause, and ownership transfer.

Attestation fields: claim ID, subject address, evidence hash, result hash, policy hash, verifier ID, confidence, verdict, created time, expiry time, and revoked flag.

Trust model: the contract does not say that a verifier is infallible. It records which operator was authorized, which verifier ID was used, and which manifest or version hashes identify the pipeline.

Networks: X Layer Testnet is chain ID 1952 with RPC <https://testrpc.xlayer.tech/terigon>. X Layer Mainnet is chain ID 196 with RPC <https://rpc.xlayer.tech>. Local Hardhat is for tests only and must not be presented as X Layer.

11 / OPERATIONAL RECORD

Database and data model

PostgreSQL is the source of truth for application operations. The chain is the source of truth for the attestation actually anchored. The records link through claim ID, result hash, contract address, chain ID, and transaction hash.

Model	Stores
VerifierProfile	Operator address, manifest, pipeline and policy versions, active status, counters, and heartbeat.
Claim	Claim text, subject address, policy ID/version, current status, and timestamps.
EvidenceItem	Kind, MIME type, storage key, source hash, byte length, and claim link.
VerificationRun	Verifier, verdict, confidence, result hash, model version, policy version, limitations.
Attestation	Chain ID, contract address, transaction hash, evidence/result/policy hashes, verdict, confidence, revocation.
AuditEvent	Lifecycle event type and JSON payload, optionally linked to a run.
Challenge	Challenger, reason, status, and created/resolved timestamps.

Migration status: the Prisma schema and initial migration artifact exist. Applying the migration is blocked only by missing valid database credentials. Once Neon or corrected local credentials are available, migrate, generate, seed, and replace the in-memory runtime repository.

12 / WHO NEEDS TO SIGN IN

Accounts, ownership, and access

Public verification is the growth surface. Ownership and administration are the protected surface.

Role	Access model	MVP behavior
Public viewer	No account required.	Inspect public proof: verdict, hashes, verifier, policy, expiry, revocation, and explorer link.
Claim submitter	Connect an X Layer wallet.	Create claims, upload evidence, view runs, and manage owned claims.
Verifier operator	Registered wallet/operator address.	Run the approved pipeline and anchor attestations.

Role	Access model	MVP behavior
Contract owner	Dedicated deployment/admin wallet.	Register verifiers, pause/unpause, transfer ownership, and revoke attestations.
Challenge participant	Public identity or wallet address.	Submit a challenge with a reason; resolution becomes an audit event.

For the hackathon MVP, avoid email/password authentication. A fresh dedicated testnet wallet is enough for deployment and operator testing. Stronger role management can come later without blocking the main proof path.

13 / NON-NEGOTIABLES

Security, trust, and failure handling

- **No secrets in the browser:** AI keys, database URLs, storage secrets, and private keys remain server-side.
- **No raw evidence onchain:** only compact hashes and decision metadata are anchored.
- **No silent model failures:** invalid JSON, missing fields, unsupported verdicts, and bad confidence stop the run.
- **No blind AI authority:** the model is constrained by policy and its limitations are stored.
- **Private bucket by default:** use short-lived upload and download permissions.
- **Wallet isolation:** use a fresh testnet wallet; never use a personal mainnet wallet or expose a recovery phrase.
- **Replay protection:** one claim ID should not be anchored twice; idempotency prevents duplicate submissions.
- **Verifier controls:** only active registered operators can anchor; the owner can deactivate or pause.
- **Human review path:** NEEDS_REVIEW is a valid outcome for ambiguity or incomplete evidence.
- **URL evidence caution:** allowlists, timeouts, size limits, and SSRF protections are required.

14 / HONEST STATUS

What is already built and what is not

Area	Current state	Real integration required
UI	Working Next.js interface with black and white editorial styling.	Polish after real flow; rename CTA after integration.
Claim lifecycle	API routes and deterministic in-memory lifecycle.	Replace runtime store with Prisma repository.
Verifier	Validation, hashing, policy builder, fixture adapter, tests.	Implement Groq extraction/assessment and real parsing.
Database	Prisma schema, generated client, migration artifact.	Valid Neon/local URL; migrate and wire repository.
Storage	S3-compatible seam and storage fields.	R2 SDK, private bucket, presigned uploads, tests.
Contracts	Solidity contracts compile and local tests pass.	Deploy testnet, register verifier, save addresses.
Chain operations	Hardhat config includes Testnet and Mainnet.	Fund wallet, deploy, verify explorer transactions, add relayer.
Accounts	No production authentication yet.	Wallet ownership before claim creation/admin actions.
Quality	Typecheck, verifier tests, contract tests, production build pass.	End-to-end real evidence, AI, storage, DB, and testnet.

Current truth: the demo is a credible product skeleton. It is not the real networked product until database, storage, Groq, wallet, and X Layer Testnet are connected.

15 / DELIVERY PATH

Implementation plan to 19 August

Window	Work	Exit condition
10 Aug	Finalize provider decisions; create Neon, R2, Groq, and dedicated wallet; configure local .env.	Services reachable and runtime no longer selects fixture.
11 Aug	Add Prisma repository; run migration, generate, and seed.	Claims and runs persist across restarts.
12 Aug	Add R2 SDK, private bucket, presigned upload/download.	Real photo/PDF uploads and hashes work.
13 Aug	Implement Groq adapter and evidence extraction.	Real evidence produces validated structured assessment.
14 Aug	Connect policy engine, persistence, audit events, and failure paths.	Offchain verification works without mocks.
15 Aug	Deploy contracts to Testnet; register verifier; save addresses.	Explorer shows contracts and registry event.
16 Aug	Add relayer/attestation service and tx hashes.	Real claim creates a real X Layer attestation.
17 Aug	Add wallet ownership and public proof polish.	Owner submits; public user verifies without account.
18 Aug	Clean demo rehearsal, security review, failure tests, redaction.	Repeatable run from fresh claim.
19 Aug	Freeze build, record video, capture links, prepare submission.	Demo, docs, and eligibility evidence complete.
20-21 Aug	Buffer for fixes, mainnet preparation, and submission review.	No last-minute architecture work.

Priority rule: every task must improve evidence upload -> AI/policy verdict -> database record -> X Layer attestation -> public proof. Extra verticals and advanced accounts come after that path works.

16 / HOW WE TELL THE STORY

Demo and video narrative

The best demo is one proof moment, not a tour of every feature. Use the solar claim because it makes the evidence gap obvious.

0:00 A blockchain can verify a transaction, but it cannot see a solar panel.
 0:08 A business submits an installation photo and inspection report.
 0:18 ProofLayer keeps originals private and fingerprints the evidence.
 0:28 AI extracts facts and applies a constrained completion policy.
 0:38 The result explains what passed, what failed, and what is uncertain.
 0:48 The verdict and hashes are anchored to X Layer.
 0:58 An OKX Wallet user inspects the public proof before trusting the claim.
 1:08 ProofLayer: evidence to onchain truth.

Visual direction: black and white, sharp grid, tight typography, real upload and result screens, a real explorer transaction, no neon, no fake metrics, and no simulated chain confirmation. If a step is not live, label it as a design preview.

17 / RELEASE GATE

Definition of done

- A user can create a claim with a connected wallet.
- A user can upload a real photo and PDF to private storage.
- The system computes evidence hashes from the exact stored bytes.
- Groq receives permitted evidence and returns structured output.
- The policy engine returns VERIFIED, REJECTED, or NEEDS_REVIEW with confidence and limitations.
- The database stores claim, evidence, run, verifier, audit events, and result.
- A registered verifier anchors one attestation per claim on X Layer Testnet.
- The public proof page displays result, hashes, verifier, policy, status, and explorer link.
- A second person can inspect the proof without seeing private originals.
- Revocation and challenge paths are visible and do not erase history.
- The clean demo works from a fresh claim without fixture data or local Hardhat pretending to be X Layer.
- Submission materials show AI use, X Layer Testnet deployment, and the project's public X account.

18 / SHARED LANGUAGE

Glossary

Term	Meaning
Attestation	An anchored statement that a verification result exists, with hashes and provenance.
Claim	The statement being evaluated, such as 'the solar installation was completed.'
Evidence digest	A SHA-256 fingerprint of evidence bytes; it identifies the exact input without revealing the file.
Result hash	A hash of the canonical verification result, including decision metadata.
Policy	The explicit rule set used to decide what evidence supports a claim.
Verifier	The registered identity and pipeline that produced an assessment.
Operator	The wallet address authorized to submit attestations for a verifier ID.
Relayer	A wallet-backed service that submits a valid attestation transaction to X Layer.
NEEDS_REVIEW	A valid outcome when evidence is incomplete, ambiguous, or outside safe automated limits.
Portable proof	A result another app or person can inspect without rerunning the entire verification.
Onchain	Data stored or referenced on the blockchain.
Offchain	Data stored outside the blockchain, such as a private file or database row.

19 / FURTHER READING

References and project sources

Internal project sources:

- /home/terry/Terrylinux_Workspce/prooflayer/README.md
- /home/terry/Terrylinux_Workspce/prooflayer/docs/implementation-plan.md
- /home/terry/Terrylinux_Workspce/prooflayer/docs/worktree.md
- /home/terry/Terrylinux_Workspce/prooflayer/packages/verifier/

- /home/terry/Terrylinux_Workspce/prooflayer/packages/contracts/
- /home/terry/Terrylinux_Workspce/prooflayer/packages/db/

Official references:

- [X Layer network information](https://web3.okx.com/onchainos/dev-docs/xlayer/developer/build-on-xlayer/network-information)
<https://web3.okx.com/onchainos/dev-docs/xlayer/developer/build-on-xlayer/network-information>
- [X Layer developer tooling](https://web3.okx.com/xlayer/docs/developer/tools/overview)
<https://web3.okx.com/xlayer/docs/developer/tools/overview>
- [OKX Wallet support for X Layer](https://www.okx.com/help/okx-wallet-has-supported-x-layer-network)
<https://www.okx.com/help/okx-wallet-has-supported-x-layer-network>
- [Groq supported models](https://console.groq.com/docs/models)
<https://console.groq.com/docs/models>
- [Groq vision documentation](https://console.groq.com/docs/vision)
<https://console.groq.com/docs/vision>
- [Neon connection pooling](https://neon.com/docs/connect/connection-pooling)
<https://neon.com/docs/connect/connection-pooling>
- [Cloudflare R2 S3-compatible storage](https://developers.cloudflare.com/r2/get-started/s3/)
<https://developers.cloudflare.com/r2/get-started/s3/>
- [Cloudflare R2 presigned URLs](https://developers.cloudflare.com/r2/api/s3/presigned-urls/)
<https://developers.cloudflare.com/r2/api/s3/presigned-urls/>